

# Documentation of sandTES\_HEX

A Matlab program for recalculation of fluidized bed heat exchangers.

Theresa Brunauer, BSc

TU Wien

Institute of Energy Systems and Thermodynamics

July 2024

## Contents

|          |                                            |           |
|----------|--------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                        | <b>3</b>  |
| <b>2</b> | <b>Design of the heat exchanger system</b> | <b>3</b>  |
| 2.1      | Geometry Transversal . . . . .             | 3         |
| 2.2      | Geometry Longitudinal . . . . .            | 6         |
| 2.3      | Compressors and Recuperators . . . . .     | 8         |
| <b>3</b> | <b>Structure of the Matlab program</b>     | <b>9</b>  |
| 3.1      | Main Class hex . . . . .                   | 9         |
| 3.2      | Compressor . . . . .                       | 11        |
| 3.3      | Recuperator . . . . .                      | 12        |
| 3.4      | Thermophysical Properties . . . . .        | 13        |
| 3.5      | Input and output functions . . . . .       | 13        |
| <b>4</b> | <b>How to use the Matlab program</b>       | <b>14</b> |

## List of Figures

|   |                                                                                   |    |
|---|-----------------------------------------------------------------------------------|----|
| 1 | Schematic representation of the position for transversal heat exchanger. . . . .  | 4  |
| 2 | Schematic representation of the geometry for transversal heat exchanger. . . . .  | 5  |
| 3 | Schematic representation of the geometry for natural circulation. . . . .         | 5  |
| 4 | Schematic representation of the position for longitudinal heat exchanger. . . . . | 6  |
| 5 | Schematic representation of the geometry for longitudinal heat exchanger. . . . . | 7  |
| 6 | Schematic representation of the recuperator system. . . . .                       | 8  |
| 7 | Schematic process of the recalculation. . . . .                                   | 15 |

## List of Tables

|   |                                                                                   |    |
|---|-----------------------------------------------------------------------------------|----|
| 1 | Property description of the class <b>hex</b> . . . . .                            | 9  |
| 1 | Continuation of the property description of the class <b>hex</b> . . . . .        | 10 |
| 2 | Property description of the class <b>compressor</b> . . . . .                     | 11 |
| 2 | Continuation of the property description of the class <b>compressor</b> . . . . . | 12 |
| 3 | Property description of the class <b>recuperator</b> . . . . .                    | 12 |
| 3 | continuation of the property description of the class "recuperator" . . . . .     | 13 |
| 4 | Used material value classes. . . . .                                              | 13 |

# 1 Introduction

The following document is a supplementary explanation of the Matlab program `sandTES_hex`. It is a tool consisting of a collection of classes and functions. It can be used to program a complex heat exchanger system consisting of several modules. The aim is to provide a better understanding of its functions and applications. It is primarily designed for the recalculation of sand bed heat exchanger modules for the sandTES technology. However, due to its object orientation, it can be adapted to any other application. The process media water, air, and  $s\text{CO}_2$  are implemented. The calculations are based on the VDI Heat Atlas [1] and the FDBR manual (VAIS Verband für Anlagentechnik und IndustrieService e.V.) [2]. The general Matlab syntax can be found in the Matlab documentation [3].

Again, it is important to note that this is a recalculation, so the approximate heat flow balances and geometry must be known in advance. A possible workflow is to take the heat balances from an Ebsilon simulation and use  $kA$  values from each module to approximate a geometry. An initial recalculation can then be performed with the specified geometry. In the next step, after evaluation, the geometry can be adjusted and another recalculation performed. These steps are repeated until the input geometry yields the desired result.

## 2 Design of the heat exchanger system

The program `sandTES_hex` recalculates a heat exchanger based on an input geometry and property data of various process media. These media include  $\text{H}_2\text{O}$ ,  $s\text{CO}_2$ , or dry air. Two geometric structures, namely transversal and longitudinal, are possible. The transversal geometry represents a cross-flow heat exchanger, while the longitudinal geometry represents a parallel flow in counter- or concurrent arrangement. Evaporator options include both natural circulation and once-through systems. These classes enable the integration of multi-stage pressure systems. In addition, it is possible to include recuperators for preheating the fluidization air.

### 2.1 Geometry Transversal

The geometry characteristics of cross-flow heat exchangers are defined as follows. The geometric representation of a transversal heat exchanger module illustrates in figure 2. In addition, a schematic representation of the position of the coordinate system for the transverse system is shown in figure 1. Fluidized particles move along the x-axis.

There are a few rules to follow when changing the geometry.

- The number of horizontal tubes can only be changed in increments of two times the flow number. This ensures that the inlet and outlet are on the same side.
- For the vertical section, the division and number of tubes should be chosen so that the bed height does not exceed three metres. This is due to the limited pressure difference a blower can supply.
- The flow number should not be higher than four. With exceptions for the evaporators.

In the case of a natural circulation evaporator, it is important to note that the tubes are positioned differently in the internal sand bed. A schematic representation can be found in figure 3. Therefore, the number of vertical tubes can only be changed in increments of two times the flow number. It is also possible to select a higher flow number. The number of horizontal tubes is independent.

The geometry is defined by the following parameters.

**Outer diameter** is defined as  $d_o$

**Wall thickness** is defined as  $s$

**Inner diameter** is defined as  $d_i = d_o - 2 \cdot s$

**Tube cross-section** is defined as  $A_0 = \frac{d_i^2 \pi}{4}$

**Number of horizontal tubes** is defined as  $n_{\text{horiz}}$

**Division of the horizontal tubes** is defined as  $t_{\text{horiz}}$

**Length of the heat exchange area** is defined as  $L_{\text{horiz}} = n_{\text{horiz}} \cdot t_{\text{horiz}}$

**Cavities** at the end of the Module are defined as  $L_{\text{cavity}}$

**Length of the sand bed** is defined as  $L_{\text{bed}} = L_{\text{horiz}} + L_{\text{cavity}}$

**Number of vertical tubes** is defined as  $n_{\text{vert}}$

**Division of the vertical tubes** is defined as  $t_{\text{vert}}$

**Bed height** is defined as  $h_{\text{bed}} = n_{\text{vert}} \cdot t_{\text{vert}}$

**Flow number** is defined as  $FN$

**Number of tubes with parallel flow** is defined as  $n_{\text{para}} = n_{\text{vert}} \cdot FN$  and  
for Natural circulation  $n_{\text{para}} = n_{\text{horiz}} \cdot FN$

**Passes** is defined as  $n_{\text{passes}} = \frac{n_{\text{horiz}}}{FN}$  and  
for Natural circulation  $n_{\text{passes}} = \frac{n_{\text{ver}}}{FN}$

**Channel width** is defined as  $w_{\text{channel}}$

**Tube length** is defined as  $L_{\text{tube}} = w_{\text{channel}} \cdot n_{\text{passes}}$

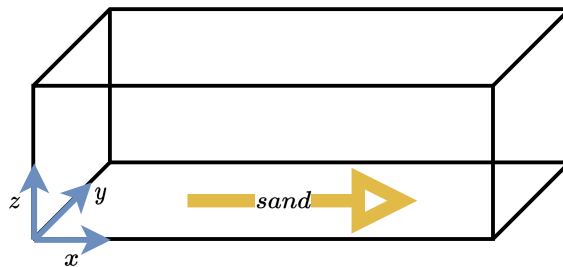


Figure 1: Schematic representation of the position for transversal heat exchanger.

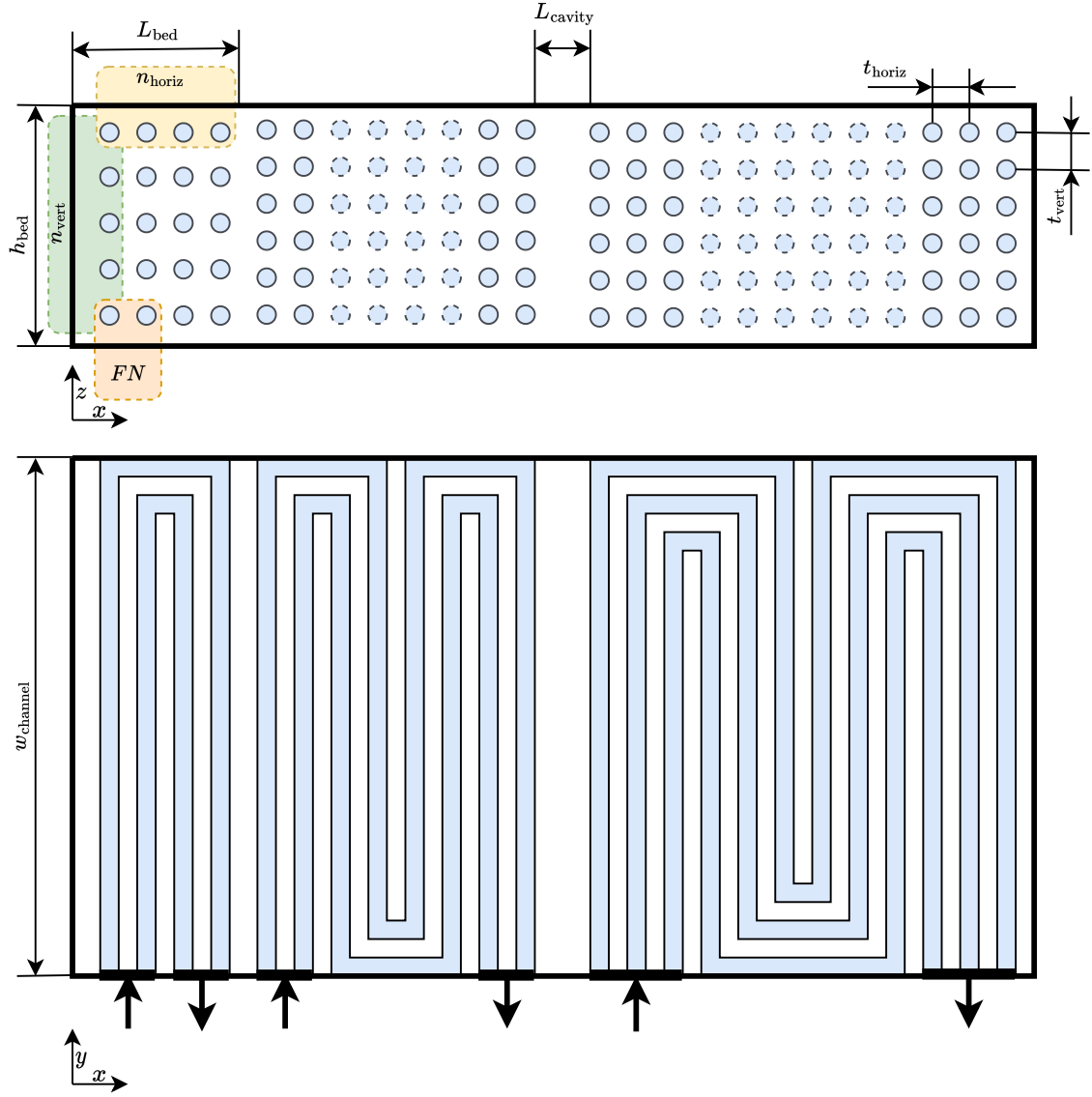


Figure 2: Schematic representation of the geometry for transversal heat exchanger.

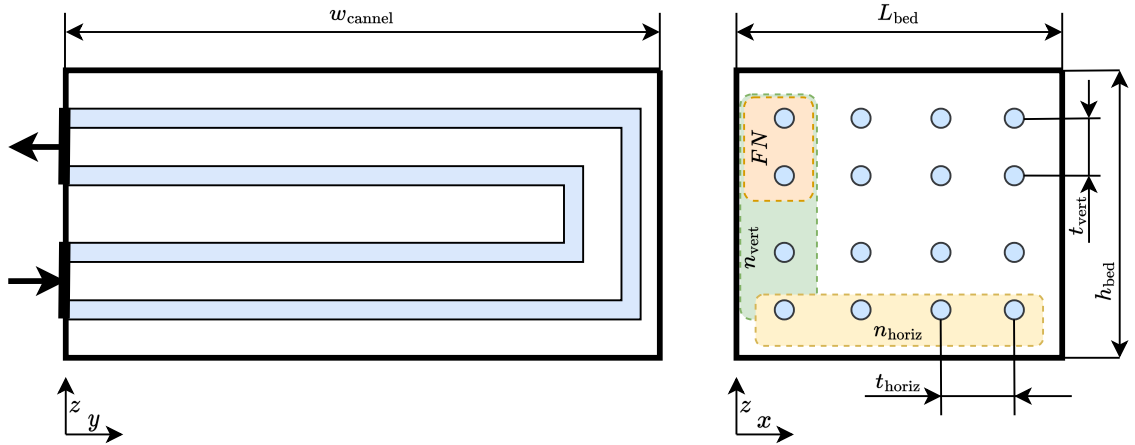


Figure 3: Schematic representation of the geometry for natural circulation.

## 2.2 Geometry Longitudinal

The geometry characteristics of parallel in counter- or concurrent flow heat exchangers are defined as follows. In order to illustrate the geometry, a schematic representation of the geometry of the heat exchanger modules is shown in figure 5. In addition, a schematic representation of the position of the coordinate system for the longitudinal system is shown in figure 4. Fluidized particles move along the x-axis.

The geometry is defined by the following parameters.

**Outer diameter** is defined as  $d_o$

**Wall thickness** is defined as  $s$

**Inner diameter** is defined as  $d_i = d_o - 2s$

**Tube cross-section** is defined as  $A_0 = \frac{d_i^2 \pi}{4}$

**Number of horizontal tubes** is defined as  $n_{\text{horiz}}$

**Division of the horizontal tubes** is defined as  $t_{\text{horiz}}$

**Length of the heat exchange area** is defined as  $L = L_{\text{channel}} \cdot n_{\text{passes}}$

**Cavities** is defined as  $L_{\text{cavity}}$

**Length of the sand bed** is defined as  $L_{\text{bed}} = L + L_{\text{cavity}}$

**Number of vertical tubes** is defined as  $n_{\text{vert}}$

**Division of the vertical tubes** is defined as  $t_{\text{vert}}$

**Bed height** is defined as  $h_{\text{bed}} = n_{\text{vert}} \cdot t_{\text{vert}}$

**Number of tubes with parallel flow** is defined as  $n_{\text{para}} = n_{\text{horiz}}$

**Passes** is defined as  $n_{\text{passes}}$

**Channel width** is defined as  $w_{\text{channel}}$

**Tube length** is defined as  $L_{\text{tube}} = L_{\text{channel}} \cdot n_{\text{passes}}$

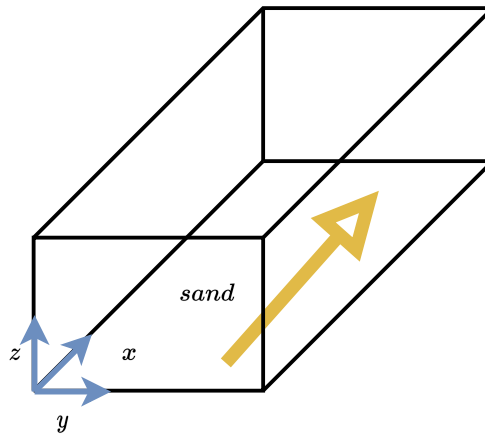


Figure 4: Schematic representation of the position for longitudinal heat exchanger.

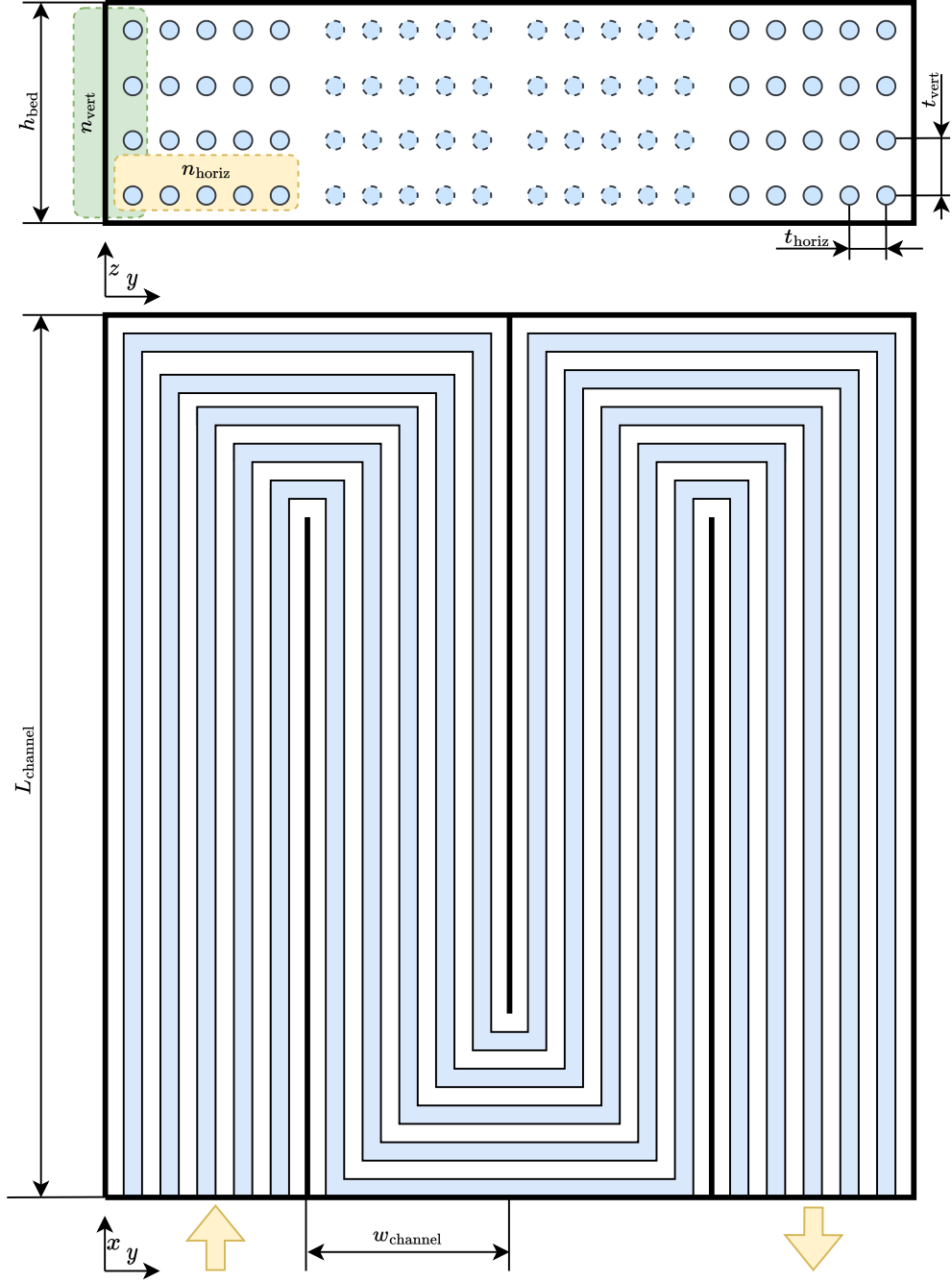


Figure 5: Schematic representation of the geometry for longitudinal heat exchanger.

## 2.3 Compressors and Recuperators

Air for fluidizing the sand bed is introduced via a fan and preheated through recuperators. The temperature of the hot exhaust air stream, which comes from different sections of the sand bed, varies depending on the number of compressors and recuperators. Figure 6 illustrates the structural layout of the individual calculation cells. As of the current status, it is only feasible to assign one or a group of heat exchanger modules to each recuperator.

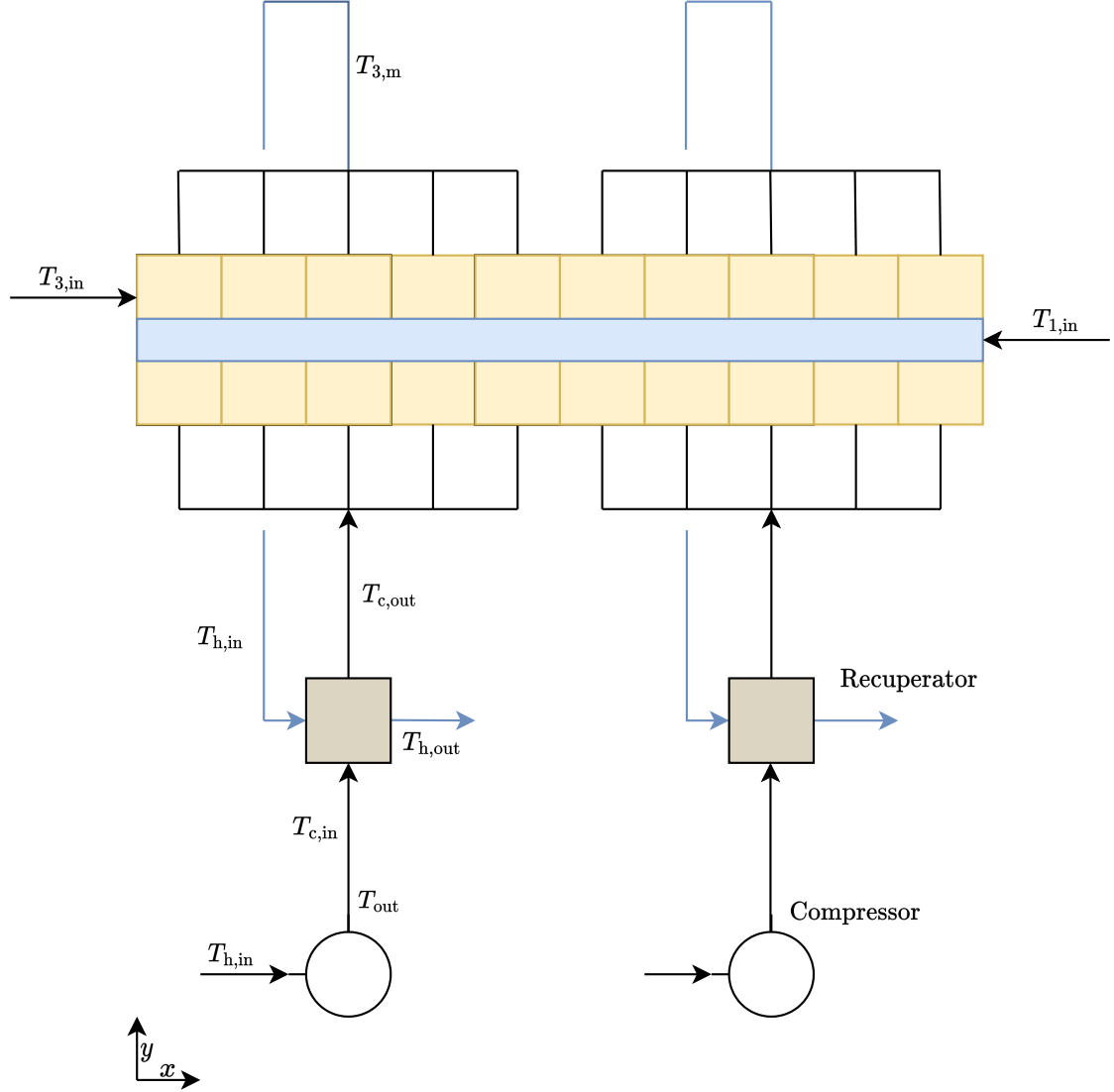


Figure 6: Schematic representation of the recuperator system.



### 3 Structure of the Matlab program

Currently, all classes are compatible with Matlab version R2023a. In addition, the Curve-Fitting Toolbox is required. The program `hex` is object-oriented and consists of several classes that are able to call and exchange values and functions. The following subsections provide an overview of the used classes and their respective functions.

#### 3.1 Main Class `hex`

The class `hex` is the basis for the calculations and drives the basic algorithm for each individual heat exchanger module. The process medium can be set via the properties. Note that the sand particle class is currently hardcoded in the class `hex`. A more detailed description of the calculation logic of `hex` can be found in the thesis *Erstellung eines Matlab-Programmes zur Nachrechnung eines Wärmeübertragers* [4].

##### 3.1.1 properties

Table 1: Property description of the class `hex`.

| property         | default value | description                                        | unit |
|------------------|---------------|----------------------------------------------------|------|
| specification    |               |                                                    |      |
| name             |               | name of the module                                 |      |
| flow_type        |               | crosscurrent or countercurrent                     |      |
| process_medium   |               | $H_2O$ , $sCO_2$ or dry air                        |      |
| prop             | IF97          | Associated material value class for process medium |      |
| charge           |               | boolean for charge or discharge                    |      |
| start conditions |               |                                                    |      |
| Qdot_start       | 0             | start value heat flow                              | W    |
| T3_start         | 0             | start value temperature sand side                  | K    |
| T1_start         | 0             | start value temperature process medium side        | K    |
| p_start          | 0             | start value pressure process medium side           | Pa   |
| mass flow        |               |                                                    |      |
| mDot1            |               | mass flow process medium                           | kg/s |
| mDot3            |               | mass flow sand                                     | kg/s |
| mDot4            |               | mass flow fluidisation air                         | kg/s |
| geometry         |               |                                                    |      |
| da               |               | outer diameter                                     | m    |
| di               |               | inner diameter                                     | m    |
| s                |               | wall thickness                                     | m    |
| l                |               | tube length                                        | m    |
| nTube            |               | number of tubes                                    |      |

Table 1: Continuation of the property description of the class `hex`.

| property               | default value | description                                               | unit                 |
|------------------------|---------------|-----------------------------------------------------------|----------------------|
| <code>npass</code>     |               | number of passes                                          |                      |
| <code>K</code>         |               | absolute roughness                                        | m                    |
| <code>nCells</code>    |               | number of cells                                           |                      |
| <code>w_channel</code> |               | width of channel                                          | m                    |
| material values        |               |                                                           |                      |
| <code>T1left</code>    |               | temperature left side                                     | K                    |
| <code>T1right</code>   |               | temperature right side                                    | K                    |
| <code>T1</code>        |               | average temperature                                       | K                    |
| <code>h1left</code>    |               | specific enthalpy left side                               | J/kg                 |
| <code>h1right</code>   |               | specific enthalpy right side                              | J/kg                 |
| <code>rho1</code>      |               | density of the process medium                             | kg/m <sup>3</sup>    |
| <code>T3left</code>    |               | temperature left side                                     | K                    |
| <code>T3right</code>   |               | temperature right side                                    | K                    |
| <code>T3</code>        |               | average temperature                                       | K                    |
| <code>h3left</code>    |               | specific enthalpy left side                               | J/kg                 |
| <code>h3right</code>   |               | specific enthalpy right side                              | J/kg                 |
| <code>p1</code>        |               | average pressure                                          | Pa                   |
| <code>delta_p1</code>  |               | pressure loss                                             | Pa                   |
| <code>p1in</code>      |               | initial pressure                                          | Pa                   |
| <code>lambda_st</code> | 50            | heat conductivity for steel tube                          | W/(m K)              |
| material values        |               |                                                           |                      |
| <code>Qdot1</code>     |               | heat flow                                                 | W                    |
| <code>Qdot_1</code>    |               | heat flow process medium side                             | W                    |
| <code>Qdot_3</code>    |               | heat flow sand side                                       | W                    |
| <code>alpha_i</code>   |               | heat transfer coefficient between tube and process medium | W/(m <sup>2</sup> K) |
| <code>alpha_a</code>   |               | heat transfer coefficient between tube and particles      | W/(m <sup>2</sup> K) |
| <code>k</code>         |               | overall heat transfer coefficient                         | W/(m <sup>2</sup> K) |

### 3.1.2 Functions

**TQ\_calc(plot)** calculates start values via heat balance. With the optional input `plot = true` a TQ-diagramm can be plotted.

**hex\_calculation(idx, nRec)** is the calculation algorithm for recalculating the heat exchanger based on the specified geometry. With `idx = 1||2` the output in the Command Window can be switched between the heat flow and the water outlet temperature. `nRec` is the number of the recuperators. This input is optional.

**plot\_Tx(type)** plots an Tx diagramm. With `type = 1||2` it can be chosen between two plot modes. `type = 1` is for one heat exchanger module and `type = 2` is for more than one different module.

**plot\_TQ(*type*)** plots an TQ diagramm. With *type* = 1||2 it can be chosen between two plot modes. *type* = 1 is for one heat exchanger module and *type* = 2 is for more than one different module.

**alpha\_in()** is an internal function used by **TQ\_calc()**. It calculates the heat transfer coefficient between the process medium and the inside of the tube. It is possible to carry out single-phase and two-phase calculations. [5],[6].

**alpha\_out()** is an internal function used by **TQ\_calc()**. It calculates the heat transfer coefficient between the sand and the outside of the tube. At the moment, this function is overwritten by a fixed default value of the heat transfer coefficient.

**deltap\_i()** is an internal function used by **TQ\_calc()**. It calculates the pressure drop over the tube. It is possible to carry out single-phase and two-phase calculations [7],[8],[9].

### 3.1.3 Static Functions

**rho(p,h)** is an internal static function, which can also output the density in the two-phase region.

**mflu(dp,rhop,p,T,A,flu)** is an internal static function that calculates the fluidization air mass flow necessary for a given degree of fluidization based on correlations for the minimum fluidization air velocity.

## 3.2 Compressor

The class **compressor** calculates the air compression for the fluidization air. The output values are temperature, pressure and the technical work. The calculation is based on the efficiency which is specified in the properties.

### 3.2.1 Properties

Table 2: Property description of the class **compressor**.

| property   | default value   | description                      | unit |
|------------|-----------------|----------------------------------|------|
| compressor |                 |                                  |      |
| pin        | 10 <sup>5</sup> | ambient pressure                 | Pa   |
| pout       |                 | pressure after compression       | Pa   |
| dp_c       |                 | pressure difference compressor   | Pa   |
| Tin        | 298.15          | temperature ambient air          | K    |
| Tout       |                 | temperature after compression    | K    |
| eta        | 0.7             | efficiency                       |      |
| wt         |                 | technical work of the compressor | J/K  |
| sandbed    |                 |                                  |      |
| h_bed      |                 | height of the sand bed           | m    |

Table 2: Continuation of the property description of the class `compressor`.

| property    | default value        | description                   | unit              |
|-------------|----------------------|-------------------------------|-------------------|
| A_bed       |                      | area of the bed               | m <sup>2</sup>    |
| delta_p_bed |                      | pressure loss of the bed      | Pa                |
| p_bed       | 0                    | pressure of the bed on top    | Pa                |
| dp          | $120 \times 10^{-6}$ | diameter of the sand particle | m                 |
| rho_p       | 2650                 | density of the particle       | kg/m <sup>3</sup> |
| eps_por     | 0.5                  | porosity of the sand bed      |                   |

### 3.2.2 Functions

**calculate(factor)** is the main function which calls all the functions necessary for the calculation of the values for the compressor and stores them in the object. The *factor* is the ratio of compressor pressure difference to bed pressure loss.

### 3.2.3 static functions

**Tout\_dp(T1, eta, p1, p2)** calculates the temperature of the air after compression.

**deltap(dp\_bed, factor)** calculates the pressure difference over the compueressor.

**w\_t(p1, p2, T1, eta)** calculates the technical work of the compressor

**pbed(pout, dp\_bed)** calculates the pressure on the bottom of the bed.

## 3.3 Recuperator

The calculation of the recuperators is based on the thermal calculation of stationary heat exchangers with fixed barriers [10].

### 3.3.1 Properties

Table 3: Property description of the class `recuperator`.

| property      | default value | description                                      | unit |
|---------------|---------------|--------------------------------------------------|------|
| input         |               |                                                  |      |
| Th_in         |               | temperature of the hot exhaust air               | K    |
| Tc_in         |               | temperature of the cold fluidisation air         | K    |
| mDot4         |               | massflow of the fluidisation air                 | kg/s |
| temp_diff_rec | 20            | temperature difference hot inlet and cold outlet | °C   |
| output        |               |                                                  |      |
| Tc_out        |               | temperature of the hot fluidisation air          | K    |

Table 3: continuation of the property description of the class "recuperator"

| property | default value | description                                                                | unit |
|----------|---------------|----------------------------------------------------------------------------|------|
| Th_out   |               | temperature of the cold exhaust air                                        | K    |
| QDot     |               | heat folw of the recuperator                                               | W    |
| kA       |               | overall heat transfer coefficient<br>multiplied with heat exchange surface | W/K  |
| NTU1     |               | dimensionless value NTU                                                    |      |
| NTU2     |               | dimensionless value NTU                                                    |      |

### 3.3.2 functions

**design(p,temp\_diff\_rec)** creates a first recuperator via NTU.

**recalculate(rec\_d,mDot4,Th\_in,Tc\_in,p)** recalculates the recuperator.

## 3.4 Thermophysical Properties

The following classes shown in table 4 are used to calculate the material value. Additional functions have been added to each class to enable the process medium to be called up in a consistent manner via the temperature and pressure specification.

Table 4: Used material value classes.

| name   | description    | created by        |
|--------|----------------|-------------------|
| IF97   | water          | Stefan Thanheiser |
| SI02   | sand           | Stefan Thanheiser |
| DryAir | air            | Stefan Thanheiser |
| CO2    | carbon dioxide | Felix Birkelbach  |

## 3.5 Input and output functions

For a standardised input and output of data, there is the option of using the following functions.

**generate\_input\_sheet (number\_Module, Module\_name, filename, storage\_location)** creates an Excel document which can be used to enter the data.

**generate\_input\_tab (number\_Module, Module\_name, filename, storage\_location)** Imports the data from the Excel file into a table in Matlab.

**output(object,name,location,type)** saves the output data in an Excel document plots a Tx and a TQ diagram and saves them. With *type* = 1||2 it can be chosen between two plot modes. *type* = 1 is for one heat exchanger module and *type* = 2 is for more than one different module.

## 4 How to use the Matlab program

A separate sequence file must be written with the correct connections of the heat exchanger modules and an outer iteration loop. Figure 7 shows the schematic structure. For a better understanding, the file **example** can be worked through.

The input can be done via an Excel document. When the **generate\_input\_sheet()** function is called, an Excel document containing a pre-defined table is created and saved. The number of modules, the module names, the desired name and location of the file must be specified in the function. The next step is to enter the specified parameters and start values into the table. The input table can be read into Matlab using the **generate\_input\_tab()** function. This requires the same input as **generate\_input\_sheet()**. A table of values is generated in Matlab. These values must be assigned to the individual objects. If not all classes, functions and input sheets are stored in the same folder, it is important not to forget to add the paths with **addpath()**.

If a natural circulation evaporator is to be recalculated, the **is\_NU** property must be set to true. In addition, the drum height, the drum pressure and a start circulation number must be specified. There is still a relatively large amount of programming that needs to be done and adapted to your system. For this reason, there is an example file to make the system easier to understand.

The next step is to generate the initial values for the outer loop calculation. This is done by calling the **TQ\_calc()** and **hex\_calculation()** functions for each module. The sand temperature **T3** is passed in each case.

A compressor object is then generated and stored in the module classes. The recuperators are then calculated with **design()** and **recalculate()**.

The outer loop of the iterative calculation starts with a while loop. It is iterated over the temperatures on the sand and process fluid sides. For natural circulation the mass flow must also be iterated. For the inner loops the functions **design()** and **recalculate()** must be called again as before. A recalculation of the recuperator is also necessary.

In the final step, the error is calculated from the change in temperature of the process medium as a condition of the while loop. If the system is a natural circulation evaporator, the error is also calculated over the change in mass flow rate.

When the while loop is finished, the data can be passed to the **output()** function.

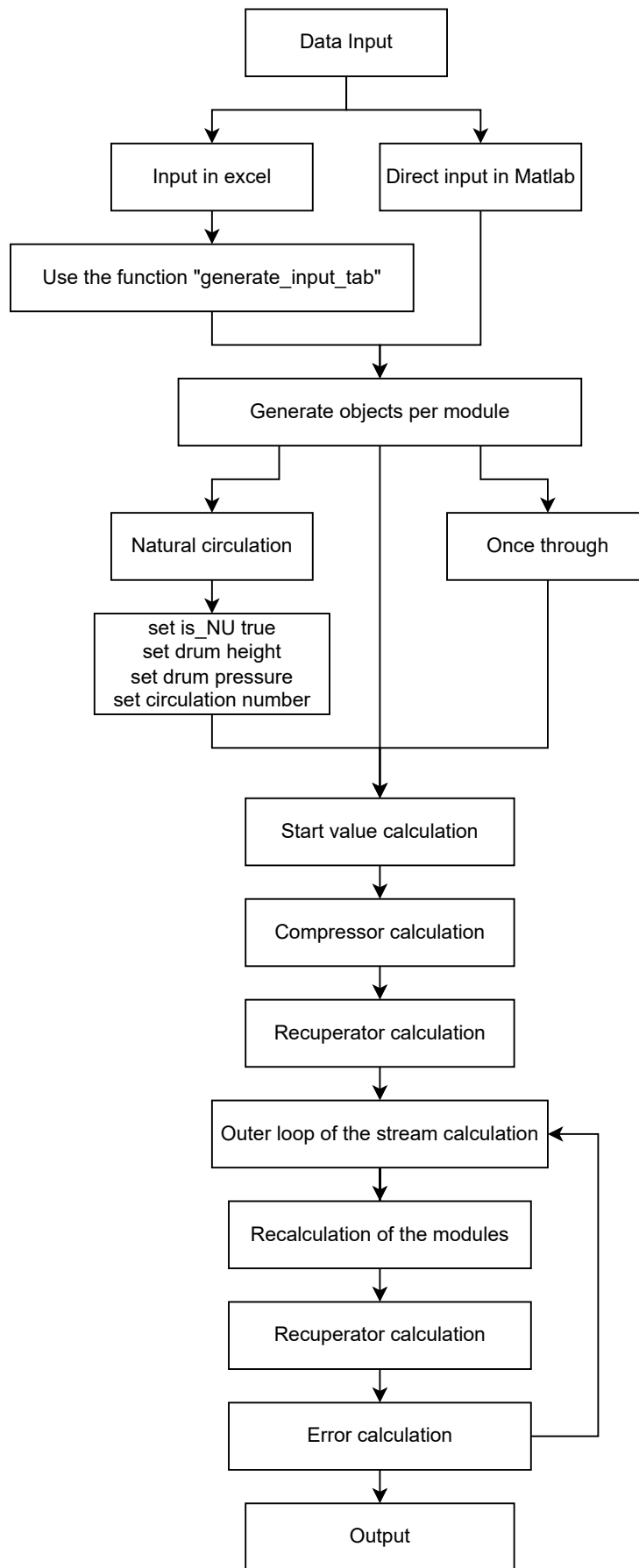


Figure 7: Schematic process of the recalculation.

## References

- [1] P. Stephan, D. Mewes, S. Kabelac, M. Kind, K. Schaber, and T. Wetzel, Eds., *VDI-Wärmeatlas* (Springer Reference Technik). Berlin, Heidelberg: Springer Berlin Heidelberg, 2020, ISBN: 978-3-662-52991-1. DOI: 10.1007/978-3-662-52991-1.
- [2] B. u. R. FDBR Fachverband Dampfkessel-, “FDBR-Druckverlust”,
- [3] *Documentation - MATLAB & Simulink - MathWorks Deutschland*. [Online]. Available: <https://de.mathworks.com/help/>.
- [4] Brunauer Theresa, “Erstellung eines Matlab-Programmes zur Nachrechnung eines Wärmeübertragers”, 2022.
- [5] M. Kind and Y. Saito, “Strömungssieden gesättigter, reiner Flüssigkeiten”, in 2019, pp. 1–27. DOI: 10.1007/978-3-662-52991-1\_58-1.
- [6] V. Gnielinski, “Wärmeübertragung bei erzwungener Konvektion: Durchströmte Rohre”, in 2018, pp. 1–9. DOI: 10.1007/978-3-662-52991-1\_42-1.
- [7] A. Wellenhofer, “Druckabfall von Gas-Flüssigkeitsströmungen in Rohren, Leitungselementen und Armaturen”, in 2018, pp. 1–20. DOI: 10.1007/978-3-662-52991-1\_82-2.
- [8] H. Nirschl, “Druckverlust in durchströmten Rohren”, in 2018, pp. 1–8. DOI: 10.1007/978-3-662-52991-1\_75-2.
- [9] M. Kind and T. Wetzel, “Strömungssieden – Druckverlust in durchströmten Verdampferrohren”, in 2019, pp. 1–4. DOI: 10.1007/978-3-662-52991-1\_56-2.
- [10] W. Roetzel and B. Spang, “Wärmeübertrager: Berechnungsmethoden”, pp. 1–41, 2019, ISSN: 2522-8196. DOI: 10.1007/978-3-662-52991-1\_5-2.